# BUT Informatique - R4.12
# Automates et Langages
## Bases de la Théorie des langages et des automates

J. Christian Attiogbé

Janvier 2026

Nantes Université





## Plan de la suite (adaptation selon contraintes)


1. Introduction
2. Problème - mot - langage
3. Langage - mot - automate
4. Langages - Grammaires
5. Alphabet - Mot
6. Langages formels
7. Grammaires
8. Types de grammaires : Noam Chomsky
9. Automates
10. Grammaires régulières et langages réguliers
11. Expressions régulières
12. Automates finis non-déterministes et déterministes
    - Automates finis déterministes
    - Automates finis non-déterministes
13. Théorème de Kleene
14. Minimisation et autres opérations sur les automates
    - Produit d'automates
    - Composition parallèle d'automates
15. Machines à états de Mealy

# Organisation de l'enseignement et évaluation

**Enseignements** : 4 semaines de CM, TD-TP et CC.

**Documents** : en ligne (accès via Madoc).

**Encadrement** : Nicolas Hernandez, Christian Attiogbé

- sem4 : Exercices Mots, Langages [Cahier d'exos N°1]
- sem5 : Exercices Langages, Grammaires [Cahier d'exos N°2]
- sem6, 7 : Exercices Automates [sur cette partie, quelques travaux de programmation en kotlin]

Evaluations en continu, TD+TP et Amphi





# Introduction

Cahier de charges

↓

Exigences → Spécification → Logiciel

**Étapes de développement**

**Analyse** - **Conception** - **Développement**+Test - Maintenance

# Problème - algorithme / Mot - langage

**Mots, langages, problèmes et algorithmes** → cœur des fondements de l'Informatique et de la programmation.

Concepts étroitement liés entre eux.
Les connaître → démêler plus facilement des problèmes d'**analyse, de raisonnement et de programmation**.

Dans cet enseignement : une introduction.
A approfondir à l'aide de la littérature abondante (ouvrages, cours, mooc, logiciels, ...).

**Théorie des Langages, Automates et Compilation** : domaines, **rigoureusement étudiés et maîtrisés**.
Nombreuses applications, base de nombreux outils.





# Problème - mot

**Problème - mot - langage**

Quelle est la relation entre les deux questions suivantes ?

1. Est-ce que **17 est paire** ?
2. Est-ce que **un entier n est paire** ?

(1) est une instance de la (2) qui en a de nombreuses.
En Informatique, problème = la donnée d'une entrée appartenant à un ensemble, et d'une propriété/question.
Résoudre problème = décider si l'entrée vérifie la propriété = savoir si elle est élément du sous-ensemble d'éléments ayant la propriété.

# Problème - mot

Si le problème est résolu par une machine (ordinateur), celle-ci va considérer, pour traiter le problème, une **représentation/encodage** de l'entrée (comme un **mot**).

**Exemple**

Problème

- n=17
- n est-il impaire ?

Décision

17 est encodé par 10001. Oui, 17 est impaire
10001 fait partie de l'ensemble des entiers impairs ;
leur représentation binaire se termine par 1 ;
c'est-à dire, ils sont tous codés par **(0|1)*1**





# Résolution de problème - Algorithme

En général, en Algorithmique/Programmation, on répond plutôt à la question (2).
Décision : on répond **oui** ou **non**.

Résoudre un problème = reconnaître si un mot donnée (entrée du problème) est élément de l'ensemble des mots satisfaisant la propriété/question.
Dans ce contexte,
un **ensemble de mots** = **langage formel** ;
on dit aussi : un langage **engendre** un ensemble de mots.

**Exemple de mot**

```
Pgm=product(a,b){intp=a;inti=0;while(i<=b)[aaachoum!]{i++;p=p+a}}
```

# Résolution de problème - Algorithme

**Exemple de mot !**

```
def product(a,b)
    { int p=a;
       int i=1;
       assert(···)
       while(i<=b) {
       p=p+a
       i++;
       ...
       }
    }
```

☞ écrit en conformité avec un langage (= ensemble de mots)





# Mot - langage - automate

## Mot - langage - automate

Un **automate est une machine** (ou un algorithme, ou une procédure) qui permet de **reconnaître les mots engendrés** par un langage / **décider** si un mot appartient à un langage.

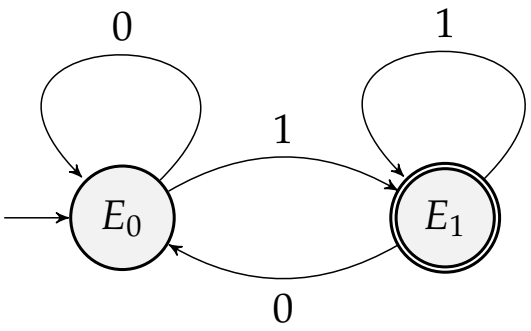



Figure: Exemple d'Automate (algorithme, impair)

# Mot - langage - automate

**Exemple**

**Problème**

- *Pgm*=`product(a,b){int p = a; int i= 0; while (i ≤ b)[aaachoum ![]{i++; p=p+a}}`
- Pgm est-il syntaxiquement correct en kotlin ?

**Décision**
Le compilateur (= algorithme qui résout le problème) décide si **le mot** *Pgm* ci-dessus **fait partie d'un des mots engendrés par le langage** kotlin.





# Quelques familles de problèmes

Des problèmes de **calcul ou d'évaluation** ; ceux pour lesquels on effectue un calcul/traitement des données en entrée ;
**Exemples :** factorielle de $n$, maximum d'un ensemble de nombres, tri d'un ensemble de données, ...

Des **problèmes de décision** ; ceux pour lesquels on peut répondre par oui ou par non ? (une entrée a une propriété)
**Exemple :** un nombre est-t-il premier ? deux nombres sont-ils premiers entre eux ? une structure est-elle ordonnée ? etc

Des **problèmes d'énumération** ;
Exemple : tous les sous-ensembles d'un ensemble d'éléments, etc

Des **problèmes d'optimisation** ; améliorer la solution d'un problème

# Alphabet

## Alphabet

Un alphabet (en théorie des langages) est un **ensemble fini** non vide de **symboles**.

Notation $\mathcal{A}$ l'alphabet ou $\Sigma$
Terminologie : **Symbole = lettre, caractère, signe**

## Exemples d'alphabet:

- $\mathcal{A} = \{0, 1\}$
- $\mathcal{A} = \{a, z, i\}$
- $\mathcal{A} = \{-,\ ;,\ :,\ =,\ ],\ [,\ ),\ (\}$
- $\mathcal{A} = \{co, ri, ca, ci\}$
- $\mathcal{A} = \{end, begin, while, done, break, wait\}$
- $\mathcal{A} = \{do, re, mi, fa, sol, la, si\}$





# Mots

## Mot

Un **mot** $w$ est une **suite finie d'éléments définis sur un alphabet** $\mathcal{A}$ donné. Un mot est donc défini sur $\mathcal{A} \times \mathcal{A} \times \mathcal{A} \times \cdots$ (on verra la notation appropriée $\mathcal{A}^*$ dans la suite)

Le **mot vide** est autorisé et est noté $\varepsilon$

## Exemples de mots

- 000001 011011001 1010101010 111 10001
- iza azi aziza izazi iiizzzaaaaaa zazizaza
- ;-) :=) ):-() [::( ;:-( :-) )))) [-)] ;-]
- cocorico cacorici caricoco cacirica
- begin end end end begin
  while begin end wait while begin
  done break while end while while end end

# Mots

## Longueur d'un mot

La **longueur d'un mot** $w$ est le nombre de symboles qui composent le mot ; c'est **un entier nul ou positif**.
On note $|w|$ la longueur de $w$, et $|\varepsilon| = 0$

## Ensemble de mots

$\mathcal{A}^*$ représente **les mots de longueur quelconque y compris le mot vide** (ens. des mots de longueur >= 0)

$$\mathcal{A}^* = \{w \mid \exists n \in \mathbb{N} \wedge w \in \mathcal{A}^n \} \quad (n \geq 0)$$

$\mathcal{A}^+$ représente **les mots constitués d'au moins un symbole** ; ce sont les mots de longueur supérieure ou égale à 1.

$$\mathcal{A}^+ = \{w \mid w \in \mathcal{A}^* \; et \; |w| \geq 1\} \qquad \mathcal{A}^* = \mathcal{A}^+ \cup \{\varepsilon\}$$





# Concaténation, préfixe, suffixe, facteur

## Concaténation, préfixe, suffixe, facteur

Etant donné deux mots, $w_1$ et $w_2$, on peut concaténer $w_1$ et $w_2$ pour former un mot $w$ constitué de $w_1$ **suivi de** $w_2$, et de longueur $|w| = |w_1| + |w_2|$.

## Notation

L'opération de **concaténation** est souvent notée . comme $w_1.w_2$
Mais on peut l'omettre quand il n'y a pas d'ambigüité, comme $w_1w_2$

## Exemples

La concaténation :

- **jour.bon** donne le mot **jourbon**
- **bon.jour** donne le mot **bonjour**
- **bonjour.**$\varepsilon$ donne **bonjour**

# Propriétés de la concaténation

## Propriétés

La **concaténation** peut se faire à droite ou à gauche d'un mot.
Mais la concaténation **n'est pas commutative**. Elle **est associative**.
$\varepsilon$ est **un élément neutre** pour la concaténation.
On dit que $(\mathcal{A}, .)$ forme un *monoïde*.
On peut concaténer un même mot $w$ autant de fois (en le précisant) qu'on veut ; par exemple, si $w$= til, alors $w^3 = (w)^3$= tiltiltil

## Exemples

jour.bon ≠ bon.jour
(bon.jour).bahia = bon.(jour.bahia)





# Préfixe - suffixe de mots

## Préfixe

On peut extraire les **préfixe** et **suffixe** d'un mot, en précisant leurs longueurs.
On peut décider si un mot est préfixe ou un suffixe d'un autre.

## Préfixe

Un mot $p$ est **préfixe** d'un mot $w$ sur un alphabet $\mathcal{A}$ s'il existe un mot $r$ tel que $p.r = w$

## Suffixe

On définit le **suffixe** de façon similaire.

Les chaînes de caractères ou *string* en programmation sont des exemples triviaux de mots.

# Langage formel

Un **langage** permet d'**utiliser** et **composer des mots** pour exprimer des idées, des intentions, des concepts, des réalités, presque tout. On distinguera la formation/structuration des mots et des expressions (la syntaxe), de leur sens (sémantique).

## Langage

Un langage $\mathcal{L}_A$, défini sur un alphabet $\mathcal{A}$, est un sous-ensemble de mots de $\mathcal{A}^*$ :

$$\mathcal{L}_A \subseteq \mathcal{A}^*$$

$$0010100111 \in \{0,1\}^*$$
$$\{\, 0111, 010100, 0101011 \,\} \subset \{0,1\}^*$$





# Opérations sur les langages

Les langages étant des ensembles de mots, on définit simplement plusieurs opérations ensemblistes sur les langages.

## Union de langages

L'**union** de deux langages $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à $\mathcal{L}_1$ ou à $\mathcal{L}_2$ :

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \vee w \in \mathcal{L}_2\}$$

## Intersection de langages

L'**intersection** de deux langages $\mathcal{L}_1$ défini sur un alphabet $\mathcal{A}_1$ et $\mathcal{L}_2$ défini sur $\mathcal{A}_2$ est le langage défini par l'ensemble des mots appartenant à la fois à $\mathcal{L}_1$ et à $\mathcal{L}_2$ :

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{w \mid w \in \mathcal{L}_1 \wedge w \in \mathcal{L}_2\}$$

# Opérations sur les langages (suite)

## Différence de langages

$$\mathcal{L}_1/\mathcal{L}_2 = \{w|w \in \mathcal{L}_1 \wedge w \notin \mathcal{L}_2\}$$

## Produit de langages

$$\mathcal{L}_1.\mathcal{L}_2 = \{w \mid \exists m \in \mathcal{L}_1 \wedge \exists v \in \mathcal{L}_2 \wedge w = m.v\}$$

## Exemple

$$\{a,b\}.\{0,1\} = \{a0,a1,b0,b1\}$$





# Opérations sur les langages (suite)

## Quotient de langages

$\mathcal{L}_2$ par $\mathcal{L}_1$

$$\mathcal{L}_1^{-1}.\mathcal{L}_2 = \{v \mid \exists m \in \mathcal{L}_1 \wedge m.v \in \mathcal{L}_2\}$$

## Exemple

$$\{a,b\}^{-1}.\{a0,a1,b0,b1\} = \{0,1\}$$

# Opérations sur les langages (suite)

## Puissance de langages

Puissance de $\mathcal{L}$ : elle est définie inductivement

$$\mathcal{L}^0 = \{\varepsilon\}$$

$$\mathcal{L}^{n_{>0}} = \mathcal{L}.\mathcal{L}^{n-1}$$

## Exemple

$$\mathcal{L} = \{0,1\}$$

$$\mathcal{L}^2 = (\{0,1\})^2 = \{00,01,10,11\}$$

$$\mathcal{L}^3 = \{00,01,10,11\}.(\{0,1\}) = \{000,010,100,110,001,011,101,111\}$$





# Opérations sur les langages (suite)

## Clôture/fermeture (de Kleene) de langage

La clôture ou fermeture d'un langage est définie comme :

$$\mathcal{L}^* = \bigcup_{i \geq 0} (\mathcal{L}^i)$$

$$\mathcal{L}^+ = \mathcal{L}^n \backslash \{\varepsilon\}$$

# Opérations sur les langages (suite)

## Constats

On voudrait bien former des mots spécifiques, et faire des suites de mots spécifiques, donc faire des **phrases** = avec une structure particulière.

Cela revient à se donner des **règles de formation des mots** et des langages.





# Grammaire : introduction

Un **langage peut être contraint par des règles particulières pour structurer la suite de mots** de l'alphabet considéré.

## Grammaire

Une grammaire est un ensemble de règles qui engendre un langage.

## Structure - syntaxe

Toutes les phrases construites en respectant les règles de grammaire d'un langage **n'ont pas forcément de sens**.
On peut exprimer des phrases sans tête ni queue ; par exemple :

le berger gerbé a mergé chantant en !

# Grammaire

## Grammaire

Une grammaire (notée G) est un quadruplet $\langle T, N, S, R \rangle$ avec

- $T$ est un alphabet, appelé le **vocabulaire terminal** ; ce sont les éléments qui vont apparaître dans le langage engendré.
- $N$ est un ensemble de symboles, appelé le **vocabulaire non terminal**, qui n'apparaîtront pas dans les mots générés ; ils seront remplacés par des terminaux ; ils sont utilisés pour la génération.
- ...





# Grammaire (suite)

## Grammaire (suite)

Une grammaire (notée G) est un quadruplet $\langle T, N, S, R \rangle$ avec

- ...
- $S$ est le symbole de départ, **Start** ou **axiome** de la génération du langage. $S$ est élément de $N$. La génération des mots commencera par $S$ en utilisant des règles pour produire des transformations de $S$ et d'autres éléments de $N$ qui sont introduits.
- $R$ est un ensemble de règles dites **règles de production**.
  Les règles de production ont la forme : $L \rightarrow R$ où
  $L \in (N \cup T)^+$ et $R \in (N \cup T)^*$
  Les règles produisent, progressivement, la structure des mots du langage voulu.

# Grammaire (suite)

On appelle aussi un symbole **non terminal**, une catégorie syntaxique.

Les éléments de $R \in (N \cup T)^*$ sont appelés des **formes**.

Une forme peut être transformée en une autre en appliquant une règle de production.

Une règle de la forme $L \rightarrow T^*$ est appelée **une règle terminale** (on ne pourra plus transformer la partie droite).





# Exemple de grammaire

## Exemple de grammaire : $G_1$

$$
\begin{array}{lll}
G_1 = & \langle & \\
& T = \{x,\ ;,\ -,\ =,\ +,\ *,\ 2,\ 7\}, & \% \text{ les terminaux} \\
& N = \{V, E, P, I, OP, SI\}, & \% \text{ les non terminaux} \\
& S = P, & \% \text{ P est l'axiome} \\
& R = \{P \rightarrow I\ \ SI & \% \text{ R contient les règles} \\
& \quad SI \rightarrow ;\ I\ SI & \\
& \quad SI \rightarrow \varepsilon & \\
& \quad I \rightarrow V = V\ \ OP\ \ E & \\
& \quad V \rightarrow x & \\
& \quad OP \rightarrow + \mid - \mid * & \\
& \quad E \rightarrow 2 \mid 7 \mid V\ \ OP\ \ E & \\
& \} & \\
& \rangle &
\end{array}
$$

# Exemple de grammaire (suite)

## Grammaire : $G_2$

$$
\begin{array}{lll}
G_2 = & \langle & \\
 & T = & \{a,b,z,f,v,i,o,n,t\}, \quad \text{\% les terminaux} \\
 & N = & \{P,M,I\}, \quad \text{\% les non terminaux} \\
 & S = & P, \\
 & R = & \{P \rightarrow aMa \\
 & & M \rightarrow bIb \mid zIz \mid fIf \mid nIn \mid tIt \mid vIv \\
 & & I \rightarrow i \mid o \\
 & & \} \\
 & \rangle &
\end{array}
$$





# Exemple de grammaire (suite)

## Grammaire : SQL (un extrait)

```
query : values
    | WITH withItem [ , withItem ]* query
    | {
      select
    | selectWithoutFrom
    | query UNION [ ALL | DISTINCT ] query
    | query EXCEPT [ ALL | DISTINCT ] query
    | query MINUS [ ALL | DISTINCT ] query
    | query INTERSECT [ ALL | DISTINCT ] query
   }
  [ ORDER BY orderItem [ , orderItem ] * ]
  [ LIMIT { count | ALL } ]
```

# Exemple de grammaire (suite)

## Grammaire : SQL (suite)

```
select:    SELECT [ ALL | DISTINCT ]
 { * | projectItem [, projectItem ]* }
 FROM tableExpression
 [ WHERE booleanExpression ]
 [ GROUP BY [ ALL|DISTINCT] { groupItem [, groupItem]* } ]
 [ HAVING booleanExpression ]
```





# Dérivation

Une forme non vide ($g$) peut **être transformée par une grammaire** en une forme éventuellement vide $d$.

## Dérivation simple

On dit qu'il y a **dérivation de $g$ en $d$ en une étape**, et on note cela $g \Rightarrow d$, lorsqu'il existe une règle de la grammaire G qui permet de transformer $g$ en $d$ (en tenant compte des préfixes ou suffixes éventuels de $g$ et $d$).

## Dérivation multiple

Par généralisation, on effectue **des dérivations en une ou plusieurs étapes** (éventuellement vides): $g \Rightarrow^+ d$ ou $g \Rightarrow^* d$.

# Langage engendré par une grammaire

## Langage engendré par une grammaire

Une grammaire engendre un langage : l'ensemble des mots obtenus en **partant de l'axiome** de la grammaire et en appliquant les règles de production, donc **des dérivations**, jusqu'à ne plus avoir de non terminaux dans les mots.

Le **langage engendré par une grammaire** $G = \langle T, N, S, R \rangle$ **est l'ensemble des mots (terminaux = dans** $T$**)** obtenus par dérivations successives à partir de l'axiome de G. On note ce langage $\mathcal{L}(G)$

$$\mathcal{L}(G) = \{w \mid w \in T^* \ \wedge \ S \Rightarrow^+ w\}$$





# Langage engendré par une grammaire

## Exemple 1

les **trois** (3) mots suivants sont engendrés par la grammaire $G_1$

x=x-7;x=x+x+2;x=x+x-x-x-7

x=x-2;x=x-x-2;x=x+x-x-x*7

x=x*7;x=x*x-2;x=x*x*x*x*7; x=x-2

## Exemple 2

Les **huit** mots suivants font partie des mots engendrés par la grammaire $G_2$

aziza, abiba

aziza, anona, afifa

azoza, afofa, aviva

# Langage engendré par une grammaire

## Remarque

En **fonction de la structuration des règles** de production d'une grammaire, on a :
- **plus ou moins de restrictions** pour la construction des mots,
- et ainsi **différents types de grammaires**.

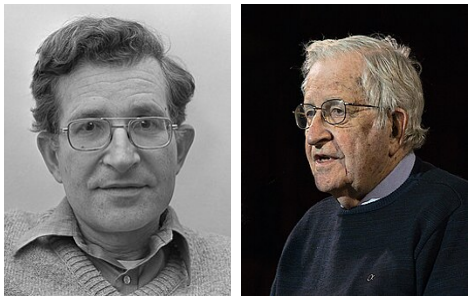


Figure: N. Chomsky (wikipedia)





# Types de grammaires

Les restrictions imposées sur la forme des règles aboutissent à une **classification et une hiérarchisation des grammaires**.

Quatre (4) types de grammaire ont été définis en 1957 par N. Chomsky.

## **Type 0** - il n'y a aucune restriction sur la structure des règles

Les grammaires de ce type permettent d'engendrer tous les langages qui peuvent être **reconnus en un temps fini par une machine** ;
ils sont dit **langages décidables**.

# Types de grammaires (suite)

**Type 1** - *Context-dependent grammar*

Ces grammaires sont dites **sensibles au contexte ou contextuelles**.
Les règles de R sont de la forme : $cXc' \rightarrow cX'c'$.
La partie gauche d'une règle spécifie un contexte de dérivation pour $X$, avec $X \in N$ et $X' \in N$, $c \in (N \cup T)^+$ et $c' \in (N \cup T)^+$

Ces grammaires engendrent les **langages contextuels**, dépendant du contexte.
Ils ne contiennent pas le mot vide $\varepsilon$





# Types de grammaires (suite)

**Type 2** - *Context-free grammar*

Grammaire hors-contexte ou **grammaire non-contextuelle**.
Les règles de R sont de la forme : $X \rightarrow X'$ avec $X' \in (N \cup T)^*$.
Elles ne spécifient pas de contexte, **le membre gauche est constitué d'un seul non terminal** ($X \in N$).

Ces grammaires engendrent les **langages non contextuels**, (indépendants du contexte, hors-contexte).

# Types de grammaires (suite)

**Type 3** - *Regular grammars* **Grammaires régulières.**

Lorsque les règles de G sont de la forme $X \to \alpha.X'$ où $\alpha \in T$ et $X, X' \in N$ la grammaire est dite régulière à droite.

Lorsque les règles de G sont de la forme $X \to X'.\alpha$ où $\alpha \in T$ la grammaire est dite **régulière à gauche**.

Les règles de la forme $X \to \alpha$ ou $X \to \varepsilon$ sont admises.

Ces grammaires engendrent les **langages réguliers** (à gauche ou à droite).
Langages réguliers = langages **rationnels** ou reconnaissables.





# Exemple de grammaire : Quel type ?

**Grammaire : $G_3$**

$$
\begin{array}{lll}
G_3 = & \langle & \\
 & T = & \{do, re, mi, fa, sol, la, si\}, \quad \% \text{ les terminaux} \\
 & N = & \{M, No, Suite\}, \quad \% \text{ les non terminaux} \\
 & S = & M, \\
 & R = & \{M \to NoSuite \\
 & & No \to do \mid re \mid fa \mid sol \mid mi \mid si \mid la \\
 & & Suite \to NoSuite \\
 & & Suite \to \varepsilon \\
 & & \} \\
 & \rangle &
\end{array}
$$

# Exemple de grammaire : Quel type ?

## Grammaire : $G_4$

$$
\begin{array}{lll}
G_3 = & \langle & \\
& T = & \{do, re, mi, fa, sol, la, si\}, \quad \% \text{ les terminaux} \\
& N = & \{M, Suite\}, \quad \% \text{ les non terminaux} \\
& S = & M, \\
& R = & \{M \rightarrow laSuite \\
& & Suite \rightarrow doSuite \mid reSuite \mid faSuite \mid solSuite \mid miSuite \\
& & Suite \rightarrow \varepsilon \\
& & \} \\
& \rangle &
\end{array}
$$





# Conclusion

## Hiérarchisation.

Les langages engendrés par ces types de grammaire sont ordonnés par une inclusion stricte. Notons $\mathcal{L}_{G_{type_i}}$ les langages engendrés par une grammaire de type $i$. On a alors :

$$\mathcal{L}_{G_{type3}} \subset \mathcal{L}_{G_{type2}} \subset \mathcal{L}_{G_{type1}} \subset \mathcal{L}_{G_{type0}}$$

Vous aurez remarqué l'importance et l'usage intensif des langages réguliers dans/pour les langages de programmation habituels et les langages de commande (Systèmes - Unix/Linux).
Les instructions (et leurs arguments) : `regexp, find, grep, ls...`

# Grammaires et Automates

Grammaires et Automates





# Grammaires régulières - Automates déterministes

$L$ est un **langage régulier** $\Leftrightarrow$ il existe une grammaire régulière (type 3) $G$ qui engendre $L$ on note $L = \mathcal{L}(G)$.
Par abus, on utilisera $\mathcal{L}$ pour un langage (s'il n'y a pas d'ambiguïté)

## Langage régulier - Automate à états finis

Tout **langage régulier** $L$ est reconnu par un **automate à états finis déterministe**.

# Grammaires régulières (rappel chap. précédent)

## Grammaire régulière à droite

Lorsque les règles de production d'une grammaire $G$ sont de la forme $X \rightarrow \alpha.X'$ où $\alpha \in T$ et $X \in N$ et $X' \in (N \cup T)$ alors la grammaire $G$ est dite **régulière à droite**. ($X \rightarrow \alpha \mid \varepsilon$ est autorisé).

On analyse une expression en la lisant de gauche vers la droite.

$$S \rightarrow \alpha.N$$

$$N \rightarrow \beta.N \mid \gamma.N \mid \eta.N \mid \cdots$$

Exemple (avec une des grammaires précédentes) :

$$do.\text{Suite} \Rightarrow do.mi.\text{Suite} \Rightarrow do.mi.fa.\text{Suite}\cdots$$





# Grammaires régulières (rappel)

## Grammaire régulière à gauche

Lorsque les règles de G sont de la forme $X \rightarrow X'.\alpha$ où $\alpha \in T$ la grammaire et $X' \in (N \cup T)$ est dite **régulière à gauche**.

On analyse une expression en la lisant de droite vers la gauche.

$$S \rightarrow N.\alpha$$

$$N \rightarrow N.\beta \mid N.\gamma \mid N.\eta \mid \cdots$$

$$\text{Suite}.re \Rightarrow \text{Suite}.fa.re \Rightarrow \text{Suite}.mi.fa.re\cdots \Rightarrow do.mi.fa.re$$

On peut effectuer **une analyse** en partant de l'axiome de la grammaire vers le mot à reconnaître, ou du mot à reconnaître vers l'axiome.

# Expressions régulières (*regular expression*)

## Expression régulière (*regular expression* - regexp)

Une **expression régulière** $E$, sur un alphabet $\mathcal{A}$ est définie inductivement comme suit (avec les 3 opérateurs . + * ) :

- $E = \varepsilon$ **ou**
- $E = \alpha$ si $\alpha \in \mathcal{A}$ **ou**
- $E = E_1 + E_2$ avec $E_1$ et $E_2$ deux expressions régulières sur $\mathcal{A}$ **ou**
- $E = E_1.E_2$ avec $E_1$ et $E_2$ deux expr. régulières sur $\mathcal{A}$ **ou**
- $E = E_k^*$ avec $E_k$ une expr. régulière sur $\mathcal{A}$ **ou**
- $E = (E_k)$ avec $E_k$ une expression régulière sur $\mathcal{A}$

exp. reg. ☞ un **mot spécifique** construit avec
les trois **opérateurs réguliers** : concaténation, union finie, itération.
L'opérateur + est encore noté |, en cas d'ambiguïté





# Expressions régulières : exemples

## Exemple 1

Expressions régulières sur $\mathcal{A}_1 = \{a, b\}$ :

a.a aaaaaaa
a+a a | a a* (a+b)
a.a.a.a.a aaaaabbaaaaaaaaa
(a+b).aaaa a+a*

## Exemple 2

Expressions régulières sur $\mathcal{A}_2 = \{li, lo, ok, ko, c1, c2, c4, pa\}$ :

lo.li.ok.pa.ko.c1

li.pa.ko.pa.ok.c1.lo

# Expressions régulières : abstractions - exemples

☞ Abstractions **des structures de contrôle** (en programmation) :

| | | |
|---|---|---|
| + ou \| | if ...then... else... | a + b |
| * | while | $(a)^*$ |
| • | séquence (;) | a.b |

$$a.((a.b)^*.c.b^*)^* \qquad a((ab)^*cb^*)^*$$

$$a.((a + b)^*.c.b^*)^* \qquad a.((a \mid b)^*.c.b^*)^*$$

$$(a.(a.b + b.a)^*.c)^+$$





# Expressions régulières = langages réguliers

**Equivalence entre expressions régulières et langages réguliers** :

**Théorème**

Toute expression régulière $E_r$ décrit un langage régulier $L_r$ :

$$E_r \implies L_r$$

**Théorème**

Tout langage régulier $L_r$ peut être décrit par une expression régulière $E_r$ :

$$L_r \implies E_r$$

# Automates à états : concepts de base

Un **automate** est une structure mathématique fondamentale, utilisée pour ‘ modéliser, raisonner, analyser, simuler : c’est un modèle

**Etat initial** ($E_i$), **état final** ($E_f$), **transition (étiquetée avec `alpha`)** entre états $E_2$ et $E_3$








# Automates à états

## Automates à états

Un **automate à états** est défini par un n-uplet : $\langle S, A, \delta, S_0, S_f \rangle$

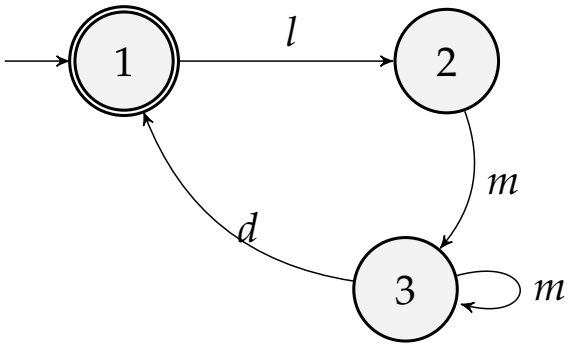
- $S$ : ensemble d’**états** fini : $S = \{S_0, \cdots, \cdots\}$
- $A$ : alphabet d’**actions** (ou **étiquettes** ou **symboles**)
- $\delta$ relation de **transition** définie comme suit $\delta : (S \times A) \leftrightarrow S$
- $S_0$ un **état initial** (avec $S_0 \in S$)
- $S_f$ un ou des **états finaux** (c’est à dire $S_f \subseteq S$)

# Automates finis déterministes (AFD ou DFA)

**Automate fini déterministe**

L'automate est **déterministe** lorsque il y a **un seul état initial** et la relation de transition $\delta$ est une fonction (et non une relation).

**Exemple d'automate déterministe ($A_0$)**





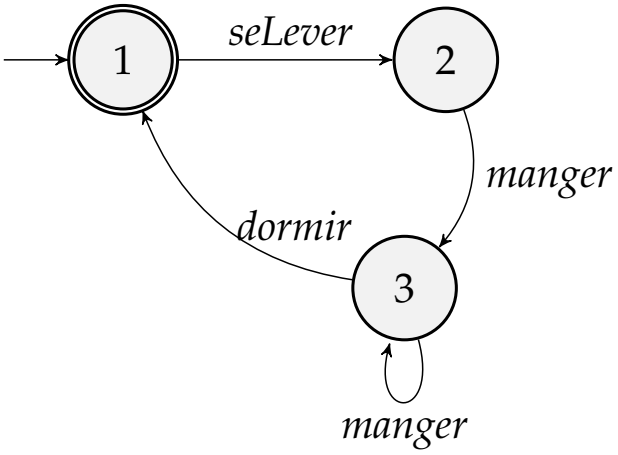



# Automates finis déterministes

L'automate $A_1$ est décrit formellement par :

$$
\begin{array}{l}
S \ = \{1,2,3\} \\
A \ = \{seLever, dormir, manger\} \\
\delta \ : (S \times A) \leftrightarrow S \text{ et} \\
\delta = \{ \ ((1, seLever), 2) \\
, \qquad ((2, manger), 3) \\
, \qquad ((3, dormir), 1) \\
, \qquad ((3, manger), 3) \ \ \} \\
S_0 \ = 1 \\
S_f \ = \{1\}
\end{array}
$$

ici, on peut noter : $\delta \ : (S \times A) \rightarrow S$
c'est une fonction



# Automates finis déterministes

On peut représenter la relation $\delta$ par une matrice :

| S \ A | seLever | manger | dormir |
|---|---|---|---|
| 1 | 2 | | |
| 2 | | 3 | |
| 3 | | 3 | 1 |

**Exercice** Représenter la relation par une matrice $(S \times S) \leftrightarrow A$

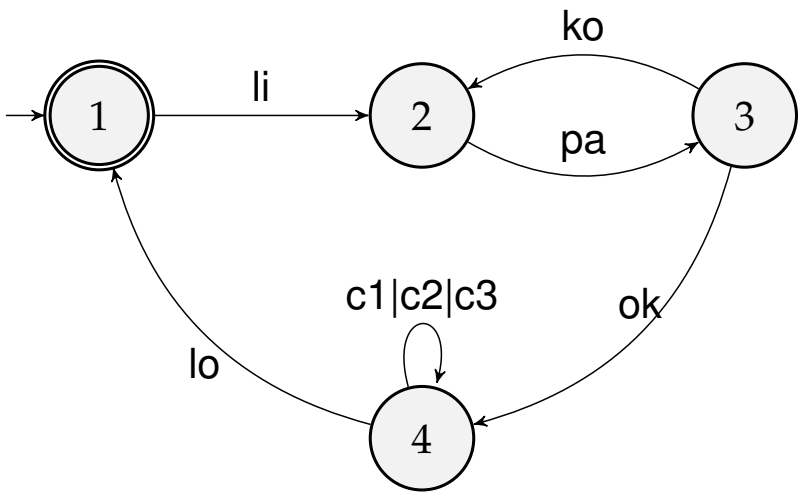
| S \ S | 1 | 2 | 3 |
|---|---|---|---|
| 1 | - | seLever | - |
| 2 | - | - | manger |
| 3 | dormir | - | manger |

☞ Quelle représentation choisir ? pourquoi ?





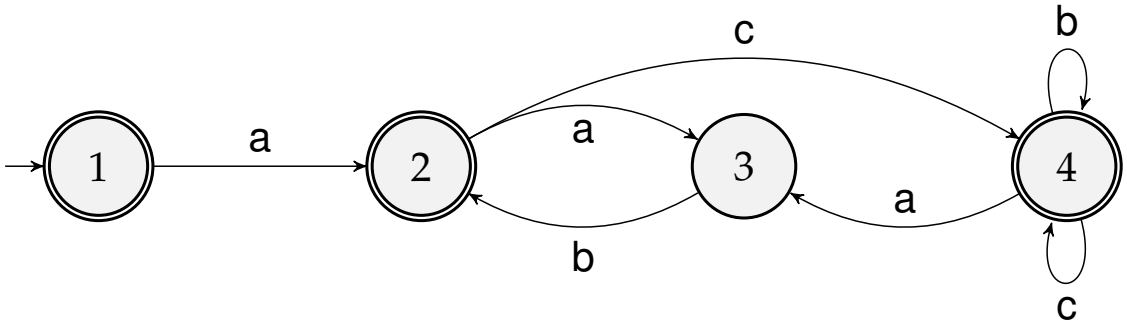
# Exemple d'automate déterministe




✌ Modélisation du comportement d'un programme !

# Exemple d'automates déterministes

Automates qui reconnaît les mots $a.((a.b)^* + c.b^*)^*$ Accepteur



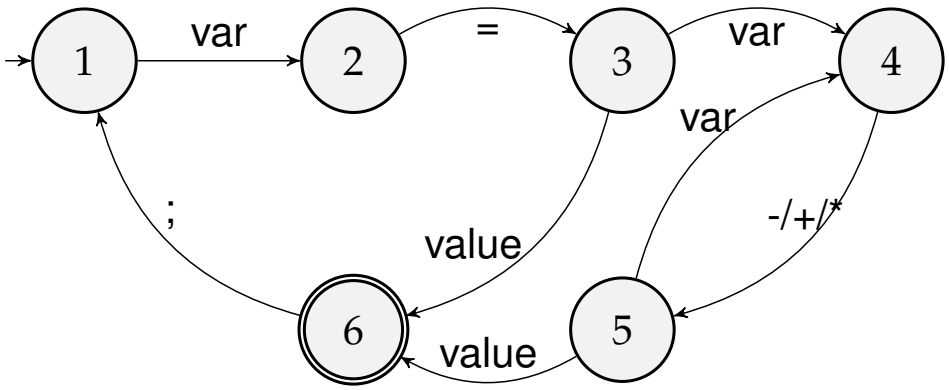

Question : Donnez quelques **mots acceptés** par cet automate

De la même façon, on peut reconnaître/analyser un programme



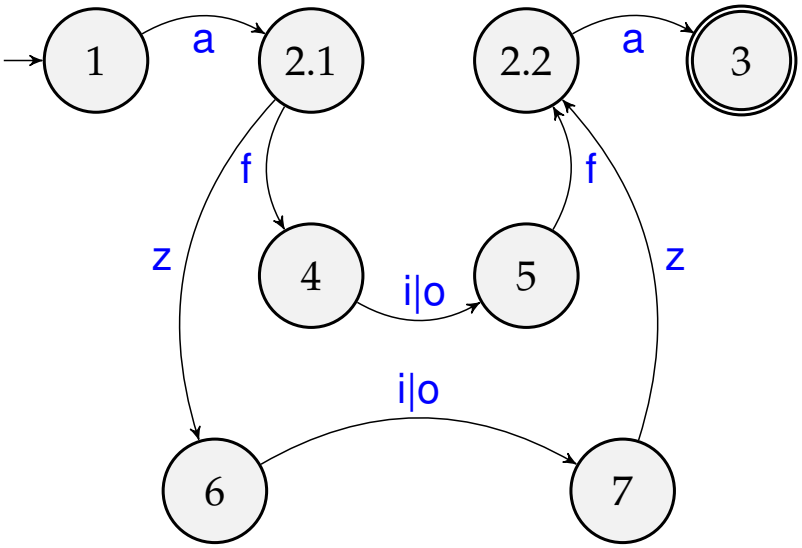


# Exemple d'automate déterministe




Automate qui reconnaît une suite d'instructions : Accepteur

# Exemple d'automate déterministe




$A = \{a, i, o, f, z\}$
$S \rightarrow a\ N\ a$
$N \rightarrow f\ I\ f$
$\mid\ z\ I\ z$
$I \rightarrow i \mid o$

Automate qui reconnaît/accepte aziza, afofa, afifa, ...





# Algorithme : comportement d'un AFD accepteur

$Etatc \leftarrow S_0$ {partons de l'état initial}
{On lit une suite/chaîne de caractères en entrée}
{On prendra carCourant comme le caractère suivant dans la chaîne}
**while** $Etatc \neq S_f \wedge Etatc \neq EtatNonDef$ **do**
  lire carCourant {on parcourt la chaîne de caractères en entrée}
  afficher (Etatc, carCourant)
  $Etatc \leftarrow \delta(Etatc, carCourant)$
**end while**
**if** $Etatc = S_f$ **then**
  Bien terminé : la suite de caractères en entrée est reconnue
**else** {On s'est arreté sur un état non défini}
  Echec : la suite de caractères n'est pas reconnue
**end if**
{Attention : conditions + algo à parfaire + 5 constituants }

# Automates finis non-déterministes (AFND)

## Automate fini non-déterministe

L'automate est **non-déterministe** lorsque :

- il a plusieurs **états initiaux** ou
- la relation de transition $\delta$ est bien une **relation** (et non une fonction) - c'est à dire, partant d'un état et d'un symbole, il y a plusieurs états suivants.

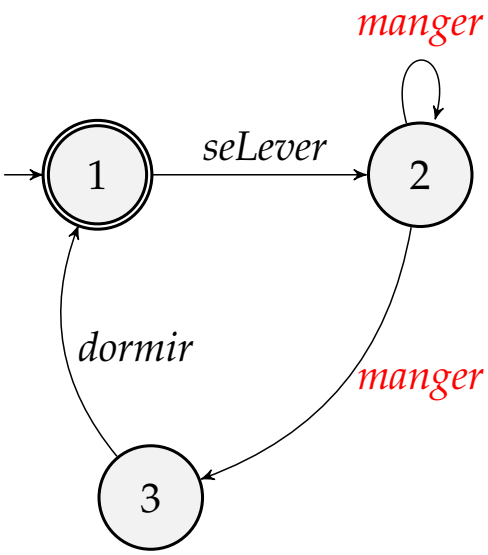
Une transition sans étiquette de l'alphabet est implicitement étiquetée par le mot vide $\varepsilon$ : il s'agit d'une $\varepsilon$-transition.

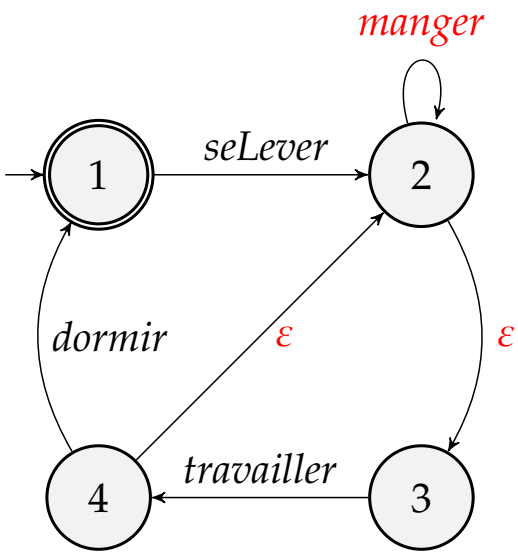
Un automate à une/des $\varepsilon$-transitions est un automate non déterministe.





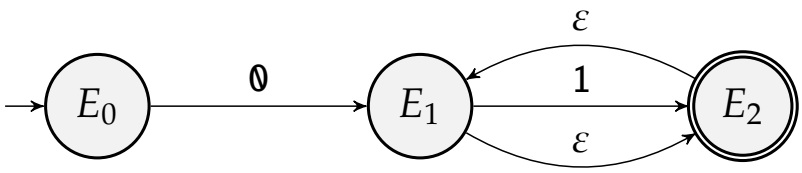
# Exemples d'automates **non-déterministes**



# Automates non-déterministes

Exemple d'un Automate ND pour reconnaître : $01^*$








# Théorème de Kleene

**Théorème 1 :** Tout langage accepté par un automate fini est régulier.

**Théorème 2 :** Tout langage régulier est accepté par un automate fini.

Le théorème s'énonce comme suit :

**Théorème de Kleen**

Soit $\mathcal{A}$ un alphabet, pour tout langage $L$ sur $\mathcal{A}^*$, (ou encore pour $L \subseteq \mathcal{A}^*$), les **deux assertions suivantes sont équivalentes** :

- $L$ est un langage régulier,
- il existe un automate fini déterministe $A$ qui reconnaît $L$, c'est à dire $L = \mathcal{L}(A)$

# Opérations sur les automates

- Minimisation d'automate : algorithme
- Produit d'automates : algorithme
- Déterminisation AFND → AFD : algorithme
- AFD → AFDN : l'algorithme de Thomson permet de construire un AFDN (avec des $\varepsilon$-transition) à partir d'une expression régulière.





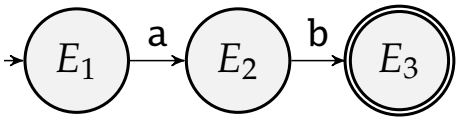
# Produit (synchrone) d'automates

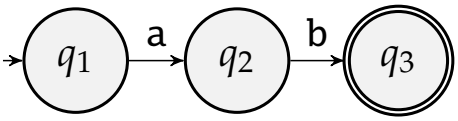
Soient 2 automates $M_1 = \langle S_1, A, \delta, S_{01}, S_{f_1} \rangle$ et $M_2 = \langle S_2, A, \delta, S_{02}, S_{f_2} \rangle$ ; $M_1$ et $M_2$ ont le même alphabet ; sinon, on effectue ($A = A_1 \cup A_2$).
Le **produit synchrone** de $M_1$ et $M_2$ est l'automate $M = \langle \cdots \rangle$ tel que :

- $S_1 \times S_2$, le produit cartésien des ensembles d'états,
- $A$, l'alphabet commun,
- $\delta_{12}$, la relation de transition (transitions communes à $M_1$ et $M_2$),
- $(S_{01}, S_{02})$, le couple des états initiaux,
- $(S_{f_1} \times S_{f_2})$, le produit des états finaux.

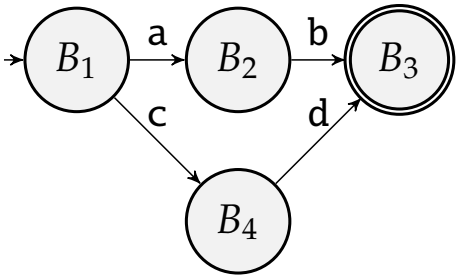
# Produit (synchrone) d'automates

La relation de transition $\delta_{12}$ est construite telle que :
$((S_{i1}, S_{i2}), \alpha, (S_{j_1}, S_{j_2})) \in \delta_{12} \iff (S_{i1}, \alpha, S_{j_1}) \in \delta_1 \wedge (S_{i2}, \alpha, S_{j_2}) \in \delta_2$



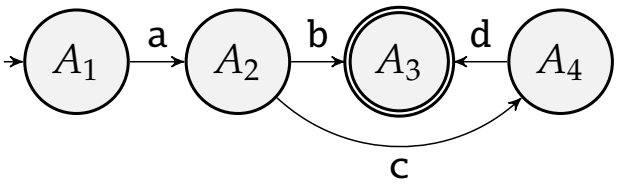

automate produit



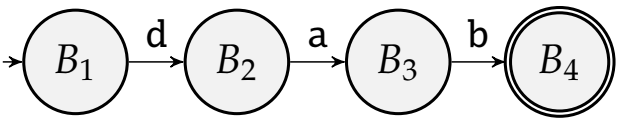





# Produit (synchrone) d'automates : exemple 2




Pas d'automate produit



# Produit synchro. d'automates - intersection de Lang.

**Remarque.** Le langage engendré par le produit de deux automates est l'intersection des langages engendrés par chacun d'eux.

$$\mathcal{L}(M_1 \times M2) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2)$$

En effet en pratique, le produit d'automates est aussi une solution à la recherche de mots reconnus **à la fois** par les deux automates.
**Exemple** : écrire un automate reconnaissant sur l'alphabet $\{A, b\}$, les mots contenant une suite de A, et ne contenant pas deux b successifs.





# Composition parallèle d'automates : algorithme

De nombreuses applications :

- interaction synchrone de systèmes/processus
- la sémantique de la concurrence entre des processus
- interaction entre un processus client et processus serveur de ressources.
- ...

Ces processus vont se synchroniser sur des **actions communes** (de leur alphabet), mais chacun d'eux fera indépendamment de l'autre les actions non communes.

# Composition parallèle de 2 automates : algorithme

Soient deux automates :
$M_1 = \langle S_1, A, \delta, S_{01}, S_{f_1} \rangle$ et
$M_2 = \langle S_2, A, \delta, S_{02}, S_{f_2} \rangle$ ;
On effectue l'union des alphabets ($A = A_1 \cup A_2$).
Le **produit parallèle** de $M_1$ et $M_2$ est l'automate $M = \langle \cdots \rangle$ tel que :

- $S_1 \times S_2$, le produit cartésien des ensembles d'états
- $A$, l'alphabet commun
- $\delta_{12}$, la relation de transition de l'interaction entre $M_1$ et $M_2$
- $(S_{01}, S_{02})$, le couple des états initiaux
- $(S_{f_1} \times S_{f_2})$, le produit des états finaux.



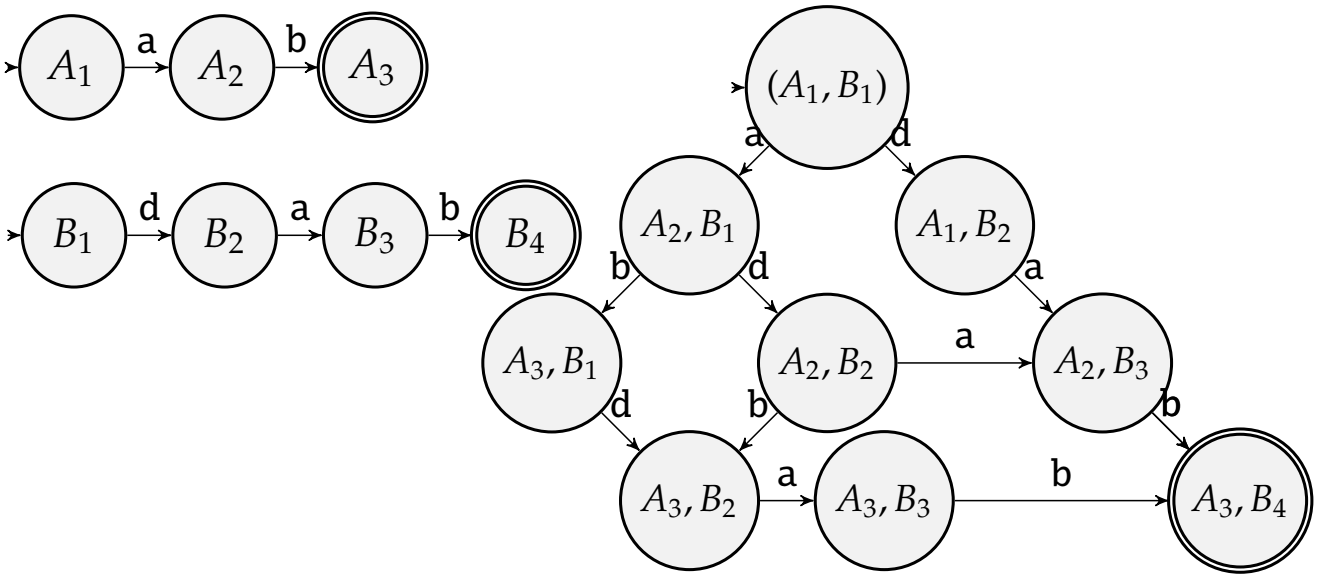


# Composition parallèle de 2 automates : algorithme

La relation de transition $\delta_{12}$ est construite telle que :
$\forall \alpha \in A$,

$$\delta_{12}((S_{i1}, S_{i2}), \alpha) = \begin{cases} (S_{j_1}, S_{j_2}) & \text{if} \quad \delta_1(S_{i1}, \alpha) = S_{j_1} \ \wedge \ \delta_2(S_{i2}, \alpha) = S_{j_2} \\ (S_{j_1}, S_{i2}) & \text{if} \quad \delta_1(S_{i1}, \alpha) = S_{j_1} \ \wedge \ \delta_2(S_{i2}, \alpha) = \text{undef} \\ (S_{i1}, S_{j_2}) & \text{if} \quad \delta_1(S_{i1}, \alpha) = \text{undef} \ \wedge \ \delta_2(S_{i1}, \alpha) = S_{j_2} \\ \text{undef} & \text{if} \quad \delta_1(S_{i1}, \alpha) = \text{undef} \ \wedge \ \delta_2(S_{i2}, \alpha) = \text{undef} \end{cases}$$

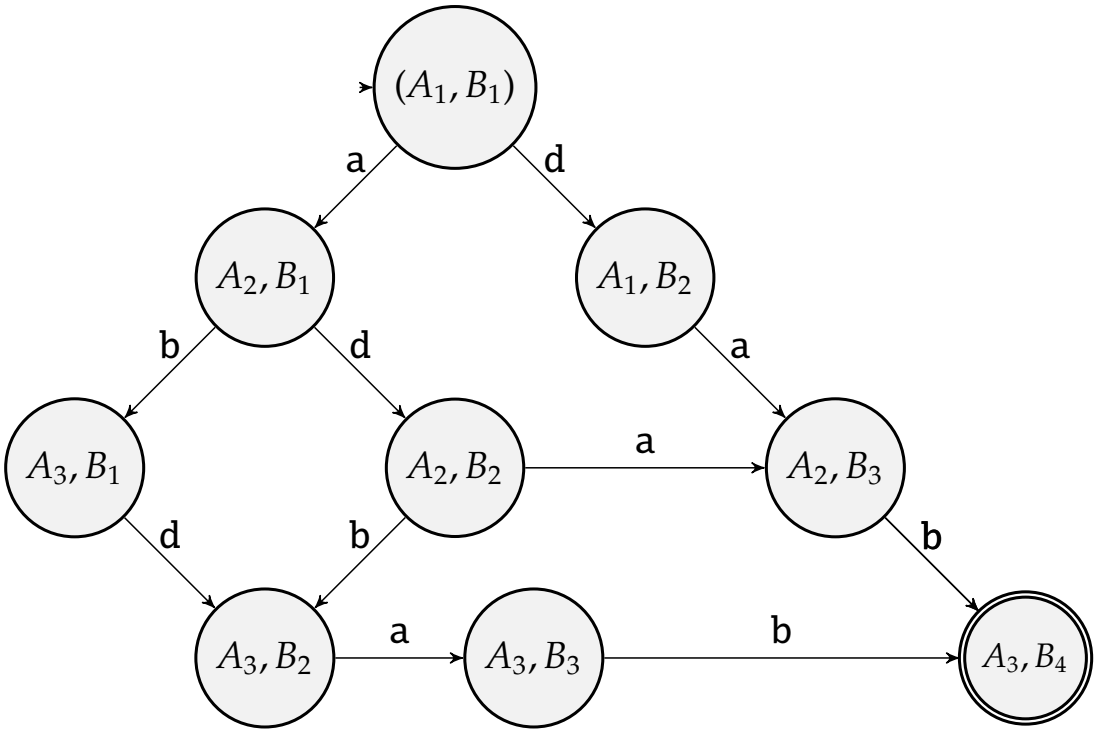
# Produit parallèle d'automates : exemple







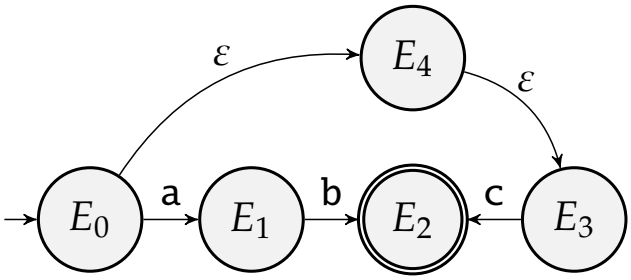

# Produit parallèle d'automates : exemple



# Algorithme de déterminisation AFND → AFD

Soit un automate non-déterministe $ND = \langle S, A, \delta, S_0, S_f \rangle$
Construction d'un automate déterministe $D = \langle S', A', \delta', S'_0, S'_f \rangle$

1. Initialisation
   $\delta' \leftarrow \{\}$
   $S'_0 \leftarrow \{S_0\} \cup \{s \in S \mid (S_0, \varepsilon, s) \in \delta^*\}$ // état initial de D :
   $S' \leftarrow \{S'_0\}$ // les éléments de $S'$ sont des sous-ensembles de $S$

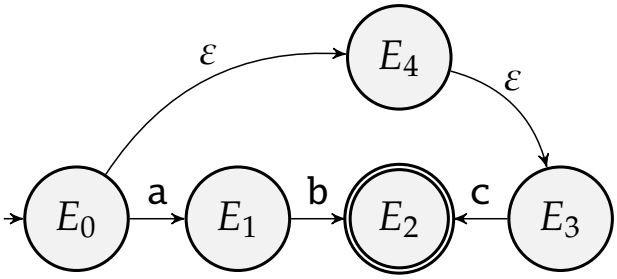


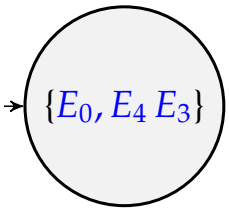

$S'_0 = \{E_0, E_4\, E_3\}$



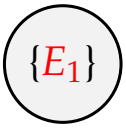


# Algorithme de déterminisation AFND → AFD



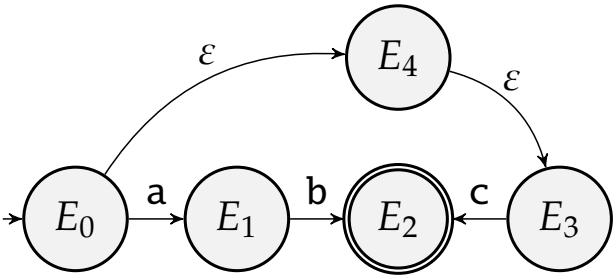



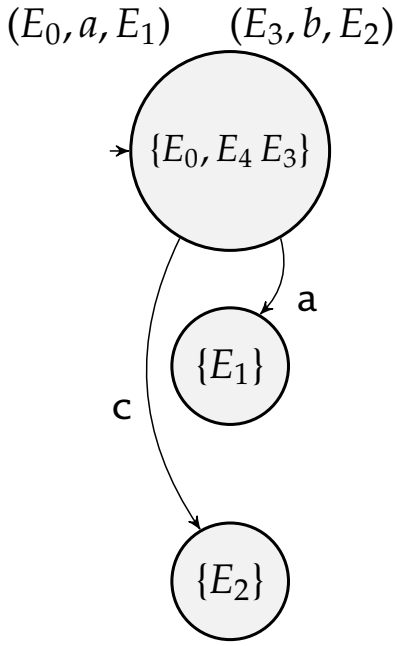

$(E_0, a, E_1)$ $(E_3, c, E_2)$






# Algorithme de déterminisation AFND → AFD

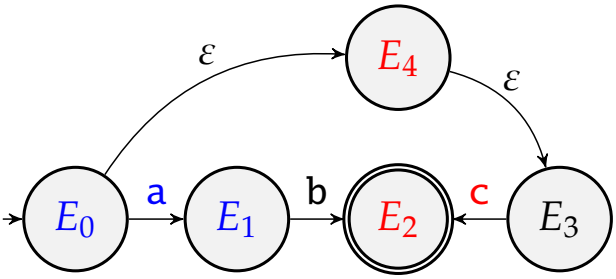


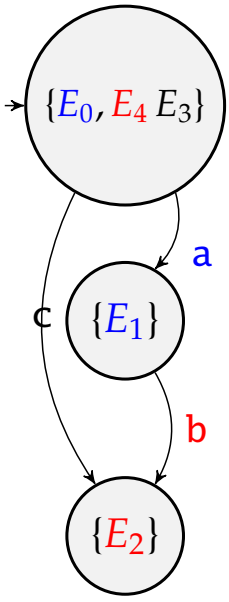





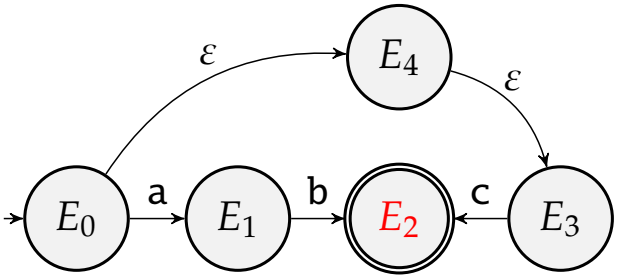


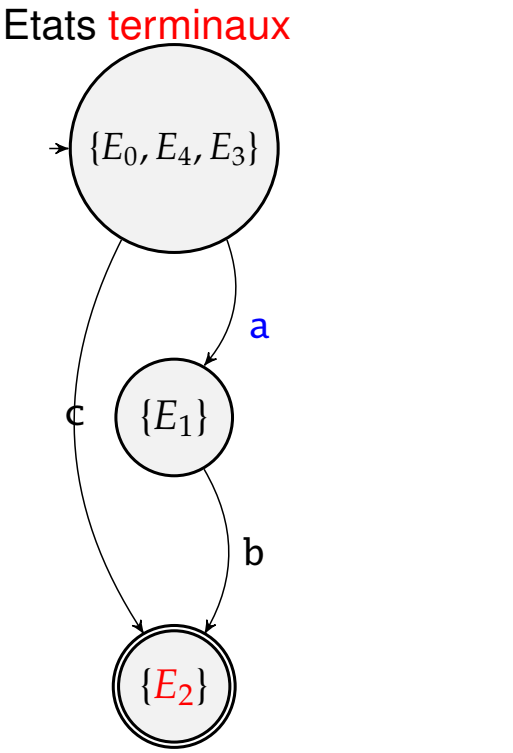

# Algorithme de déterminisation AFND → AFD



# Algorithme de déterminisation AFND → AFD








# Algorithme de déterminisation AFND → AFD (suite)

1. Construction des transitions de D

   **Pour tout** $s' \in S'$ (chaque état de D, non encore traité)

   **Pour tout** $\alpha \in A$ calcul des états atteints avec le même $\alpha$ à partir de chaque état de s', avec' $\alpha$

   $s'' \leftarrow \{p \in S \mid \exists p' \in s' \wedge (p', \alpha, p) \in \delta\}$

   **Si** $s'' \neq \{\}$ **alors** // on a un nouvel état de D, à fermer avec $\varepsilon$

   $s'' \leftarrow s'' \cup \{t \in S \mid \exists t' \in s'' \wedge (t', \varepsilon, t) \in \delta^*\}$

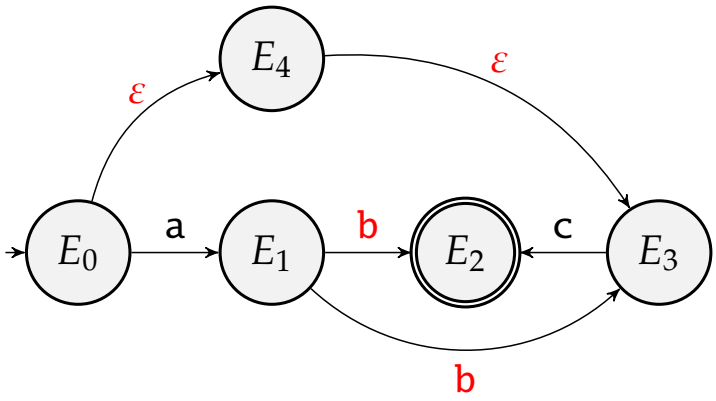
   $\delta' \leftarrow \delta' \cup \{(s', \alpha, s'')\}$ construction/maj de $\delta'$

   $S' \leftarrow S' \cup \{s''\}$ // construction/évolution de $S'$

2. Etats finaux

   $S'_f \leftarrow \{s' \in S' \mid s' \cap S_f \neq \{\}\}$ ceux de D, contenant un état final de ND

# Exemple 2 : déterminisation AFND → AFD

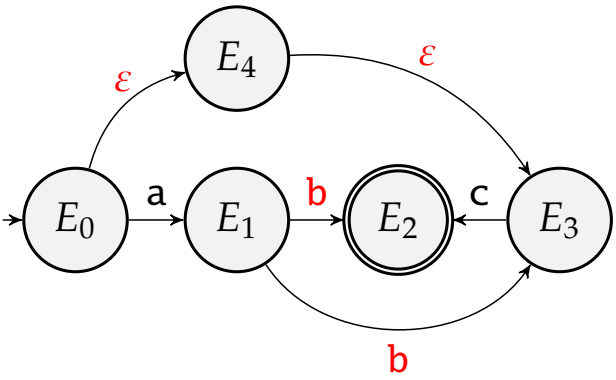


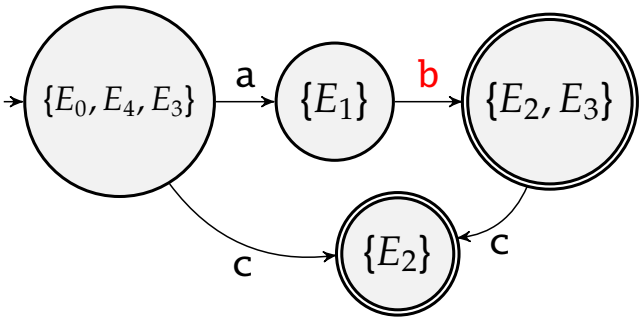

☞ **Mots** reconnus ou **Traces** de l'automate ND : c, ab, abc





# Exemple 2 : déterminisation AFND → AFD




✌ **Mots** reconnus ou **Traces** de l'automate D : c, ab, abc

# Références - opérations sur les automates

- Quelques algos `www.irif.fr/~asarin/al2k2/algos.pdf`
- Automate fini minimal `deptinfo.unice.fr/~julia/AL/03al.pdf`
- Les automates `gallium.inria.fr/~maranget/X/421/poly/automate.html`
- Automates `http://www-verimag.imag.fr/~perin/enseignement/RICM3/infaeg/cours/html/INFO3_A&G_AEF.html`





# Machine à états de Mealy

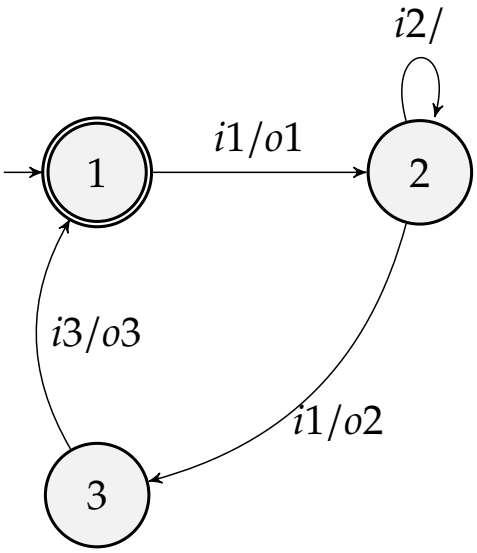
Un automate de Mealy est défini par un n-uplet
$\langle S, A = In \cup Out, \delta, \delta_o, S_0, S_f \rangle$

- Ensemble d'états : $S$
- Ensemble d'entrées : $In$
  (événements d'entrées,
  conditions)
- Ensemble de sorties : $Out$
  (actions de sortie, traitements)
- Un état initial : $S_0$
- **Deux relations(fonctions) :**
  - transition $\delta : S \times In \rightarrow S$
  - **sortie** $\delta_o : S \times In \rightarrow Out$
- Etat final $S_f$

# Machine à états de Mealy

Un automate de Mealy est défini par un n-uplet
$\langle S, A = In \cup Out, \delta, \delta_o, S_0, S_f \rangle$

- Ensemble d'états : $S$
- Ensemble d'entrées : $In$ (événements d'entrées, conditions)
- Ensemble de sorties : $Out$ (actions de sortie, traitements)
- Un état initial : $S_0$
- **Deux relations(fonctions) :**
  - transition $\delta : S \times In \rightarrow S$
  - **sortie** $\delta_o : S \times In \rightarrow Out$
- Etat final $S_f$








# Machines de Mealy : notation

- Notation des transitions : $S_s \xrightarrow{\text{i/o}} S_t$
- si l' **entrée** $i$ est reçue alors que le système est dans l'état $Ss$, la **sortie** $o$ **est produite** et le nouvel état du système est $St$
- $i$ est aussi appelé le déclencheur (*trigger*).

La **notation des transitions est étendue** de la façon suivante : les transitions entre états ont la forme evt [garde] / actions*.

evt est l'entrée reçue/lue.
[garde] : garde est une condition sur l'état du système après l'événement evt.
action* est une suite de 0 ou plusieurs actions.

# Exemple d'automate de Mealy

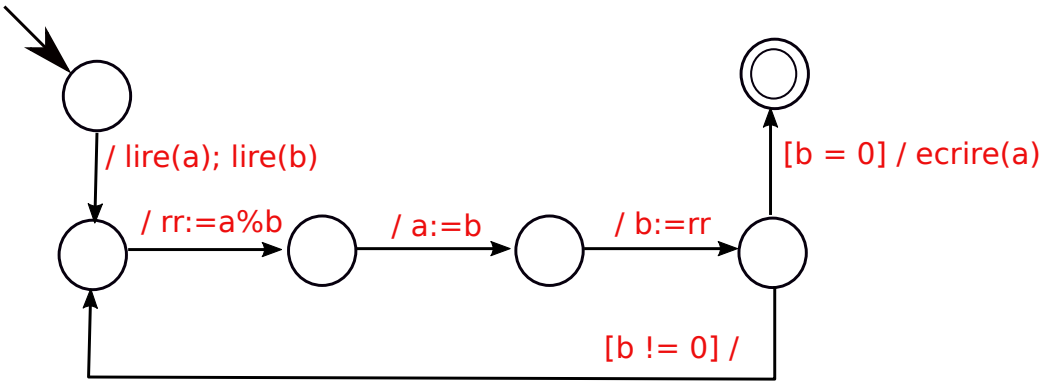



Figure: Exemple de modèle





# Caractéristiques d'une machine de Mealy

- **non hiérarchique**
- un état dénote l'état complet du système
- le système est dans **un seul état à la fois**
- une **transition est atomique** ; elle ne peut être décomposée.

# Terminologie

- **Espace d'états** : ensemble des états (possibles) d'un système
- **Relation de transition, Fonction de transition**
- Automate (*Automaton, Automata*)
- Machine à états finis *Finite State Machine*)
- **Graphe d'états**
- Diagramme Etat/Transition (*State/Transition Diagram*)
- Système de transitions (étiquetées) *Labelled Transition System*
- Espace d'états (*state space*)





# Modélisation de systèmes

En modélisation et en construction de logiciels en particulier, on **prévoit le comportement des logiciels**, à l'aide de **modèles**. Les automates (avec les graphes) sont largement utilisés comme modèles du logiciel.

Le **comportement d'un système** (quel qu'il soit), peut être vu comme une suite (transitions) des différents états du système.

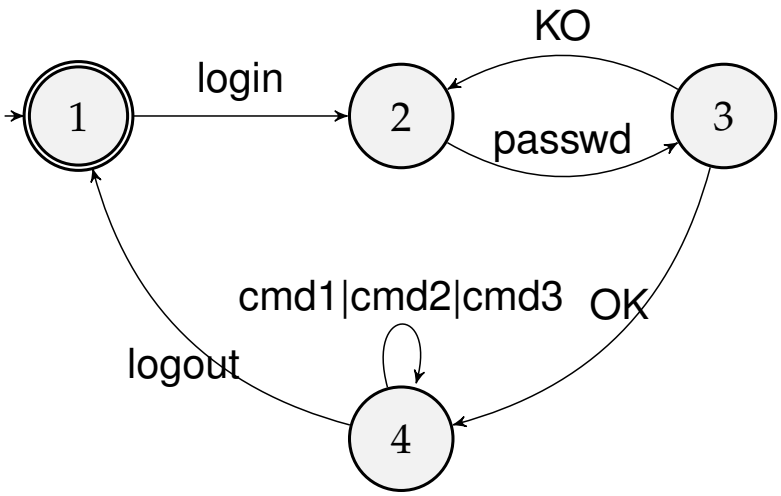
Un modèle permet de **prédire les comportements corrects et incorrects** (*bugs*) ; il faut pour cela que les modèles soient (mathématiquement) les plus précis possibles.

# Modélisation de systèmes

## Modélisation de système

Un modèle (un automate à états) simule mathématiquement un système ; il permet d'en simuler le comportement.

**Exemple** Modèle d'un processus client qui se connecte à serveur.

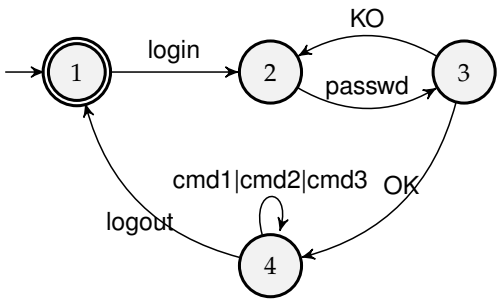







# Modélisation de systèmes

Modèle d'un **processus client (C)** qui se connecte à serveur.




**mot équivalant** :

$C = (login.P)^*$

$P = passw.Suite$

$Suite = KO.P + OK.S_1$

$S_1 = cmd1.S_1 + cmd2.S_1 + cmd3.S_1 + logout$

ou bien :

$C = (login.passw.(KO.passwd)^*.OK.(cmd1 + cmd2 + cmd3)^*.logout)^*$

# Modélisation de systèmes

## Modélisation de systèmes (Client/serveur)

Un modèle (un automate à états) qui simule mathématiquement le **comportement d'un client**.

Un modèle (un automate à états) qui simule mathématiquement le **comportement d'un serveur**.
Interaction entre les deux : produit synchrone/asynchrone

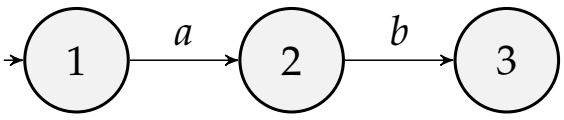
**Exemple** Modèle d'un processus client qui se connecte à serveur.
**Comportement de l'interaction entre client(s) et serveur(s)**



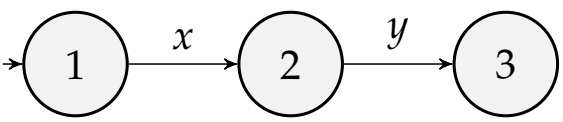


# Modélisation de systèmes

Description du comportement d'un processus (P1=**a.b**):



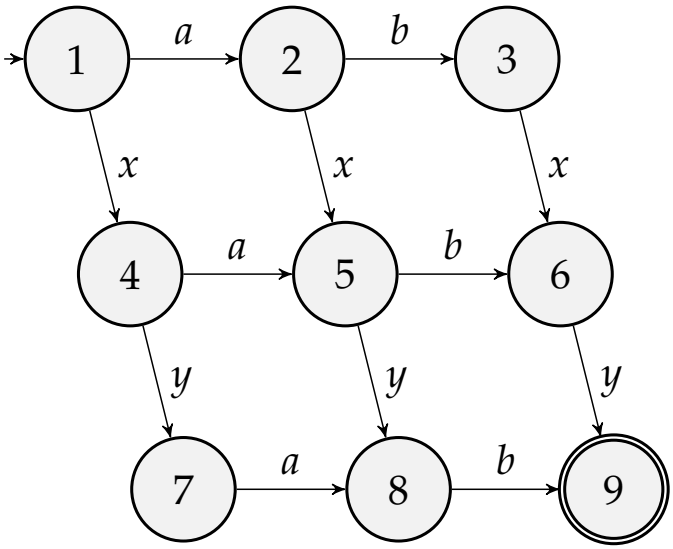

Description du comportement d'un processus (P2=**x.y**):




Comment se comporte **P1 || P2**, exécution parallèle ?

# Modélisation de systèmes

Description du parallélisme entre processus (**a.b || x.y**):

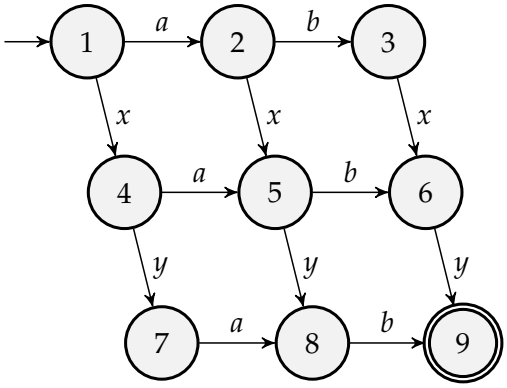







# Etude ou analyse des systèmes (logiciels ou pas)

**Traces** de :




Tous les mots : `axby, abxy, axyb, xayb, xyab, xaby, xyab` : sémantique comportementale

# Etude ou analyse des systèmes (logiciels ou non)

Etude (analyse) des propriétés d'un logiciel/système : avec les automates, les **traces** ; behavioural semantics,...
Exemples

- Détection d'anomalie : mot préfixe, mot suffixe.
- Est-ce que toutes les personnes dans un bâtiment ont badgé ?
- Génération des Tests des logiciels ou des systèmes.





# Extensions/généralisation des automates

Il y a plusieurs modèles et familles de modèles à états.
Plusieurs langages/formalismes de modélisation utilisent ce modèle.

- **System Design Language (SDL)** (normalisé par la *International Telecommunication Union* (ITU), 1988, très utilisé en Télécoms et ailleurs)
- **Statecharts** de David Harel (hiérarchiques) 1987, utilisé pas exemple dans papyrus, ...
- **Réseaux de Petri** de Carl Adam Petri (1962, 1969)
- **Algèbres de processus**, telles que *Calculus of Communicating Systems* (CCS, 1980) de R. Milner, *Communicating Sequential Processes* (CSP, 1978) de T. Hoare. C'est la famille la plus expressive des modèles à états.

# Extensions/généralisation des automates

Les états et les transitions des automates peuvent être étendus pour gagner en expressivité

- Automates à états temporisés
  (les transitions comportent des contraintes de temps)
- Automates à états probabilistes
  (les transitions comportent des données de probabilités)
- ...





# Conclusion

**Mots et Langages** : concepts fondamentaux.

**Langages et Grammaires** : formalismes de construction

**Langages et Automates** : description, modélisation, raisonnement

**Automates** : modèle fondamental en Informatique avec de nombreux algorithmes et outils

☞ Nous avons fait une introduction ; reste à approfondir

# Quelques références

de nombreuses ressources (livres, cours, sites web) .
- John E. Hopcroft, Rajeev Motwani et Jeffrey D. Ullman,
*Introduction to Automata Theory, Languages, and Computation,*
Addison Wesley, 2001, 2e éd..
- Jacques Stern
*Fondements mathématiques de l'informatique*
Ediscience international, 2003 (Chapitre 1)
- Yliès Falcone, Jean-Claude Fernandez
*Automates à états finis et langages réguliers Rappels des notions essentielles et plus de 170 exercices corrigés* Dunod, 2020





# Quelques références

- Compilateurs : Principes, techniques et outils
Alfred AHO, Ravi SETHI et Jeffrey ULLMAN.
Dunod, 2000. (vers anglaise *Compilers, Addison Wesley, 1986*)
- Alfred Aho, Monica Lam, Ravi Sethi et Jeffrey Ullman
*Compilateurs : principes, techniques et outils : Avec plus de 200 exercices*, Pearson, 2007
- Peter Linz,
*An Introduction to Formal Languages and Automata,*
Jones & Bartlett Learning, 2001, 410p.

# Quelques références

et des polys de cours
- Anca Muscholl,
https://www.labri.fr/perso/anca/Langages/cours/, Université de Bordeaux
- Gérard Duchamp, Institut Galilée
https://lipn.univ-paris13.fr/~duchamp/Enseignement/L3_Info/Supports/Cours/THL_2.pdf
- Michel Rigo, Théorie des automates et langages formels
http://www.discmath.ulg.ac.be/cours/main_autom.pdf
- Notes de révision : Automates et langages
Benjamin Monmege et Sylvain Schmitz
http://www.lsv.fr/~schmitz/teach/2013_agreg/notes-r62.pdf
- Notes de cours, de Christine Solnon
http://perso.citi-lab.fr/csolnon/langages.pdf





# Travaux pratiques

En TD, construction de plusieurs automates ; serviront en entrée d'une application à construire.
Le principe est donné dans la figure 4 ; implantation en kotlin.




Figure: Architecture simplifiée de l'application à écrire

# Travaux pratiques : menu à proposer

```
---- Nantes Université, BUT2 Informatique, 2024 ----
-- TP - Modélisation et analyse à l'aide des automates --
--- Menu de l'application (développée par par Moa TOA) ---
1. Smiley (pour reconnaître un des smileys)
2. HH:MM (pour reconnaître une heure bien formée)
3. JJ/MM/AAAA
4. Adresses électroniques
5. Polynômes
6. Expressions arithmétiques
7. Polynômes
...
99. Arrêt de l'application
Votre choix (1-99) ?
Je vous demanderez ensuite la chaîne à analyser, Merci
----------------------
```





# TP : encodage des automates déterministes

Encodage **un automate $A$ : une classe avec le quintuplet** qui définit formellement l'automate ; (en plus du nom de l'automate).
Il faudra des *getter/setter* pour chaque constituant.
**Au moins 3 classes** :

- Etat
- Symbole
- Automate

# TP : encodage des transitions

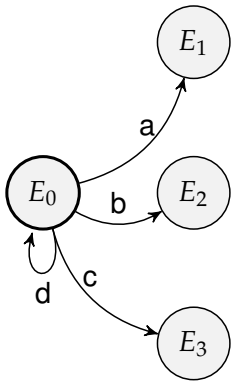



Figure: Un noeud d'un automate ...

**Exemple :** l'état $E_0$ aura comme attribut la **hashmap** outgoingTransition suivante : $\{\langle d, E_0\rangle, \langle a, E_1\rangle, \langle b, E_2\rangle, \langle c, E_3\rangle\}$





# TP : encodage des automates déterministes

- Chaque état (une classe) a parmi ses attributs les transitions sortantes et quel/s symbole/s de l'alphabet permettent d'atteindre ses transitions.
- Les transitions sortantes sont par exemple représentées par une HashMap<clé, valeur>, où la clé est un symbole et la valeur l'état atteint.
- Ainsi dans chaque état, on définit facilement une fonction qui décide si un symbole $s$ lu est une action sortante ou pas ($ss$ est parmi les clés) ; et lorsque c'est le cas, une fonction qui donne immédiatement l'état atteint : outgoingTransition(ss).